U O W

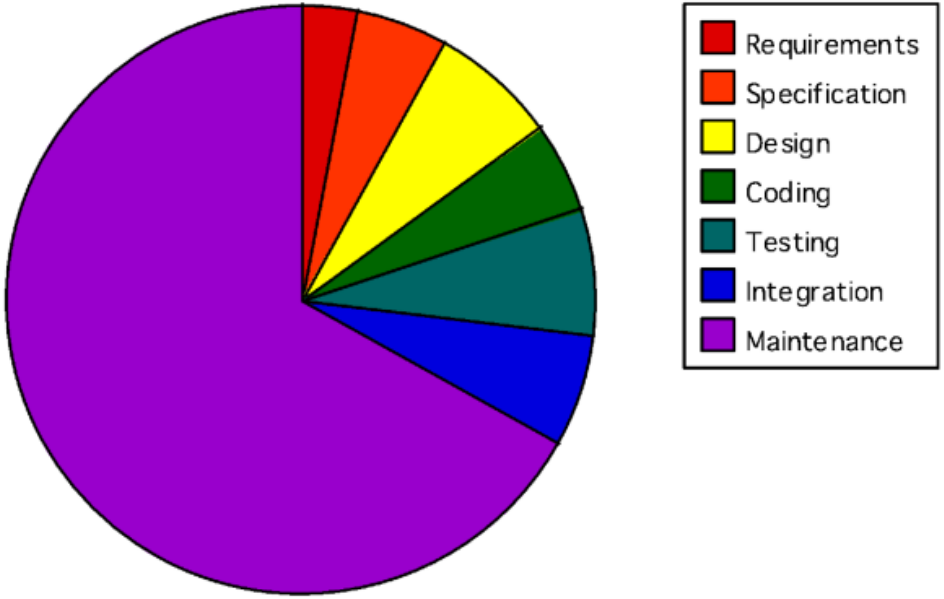
CSIT314 Software Development Methodologies

DevOps

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Development Activities

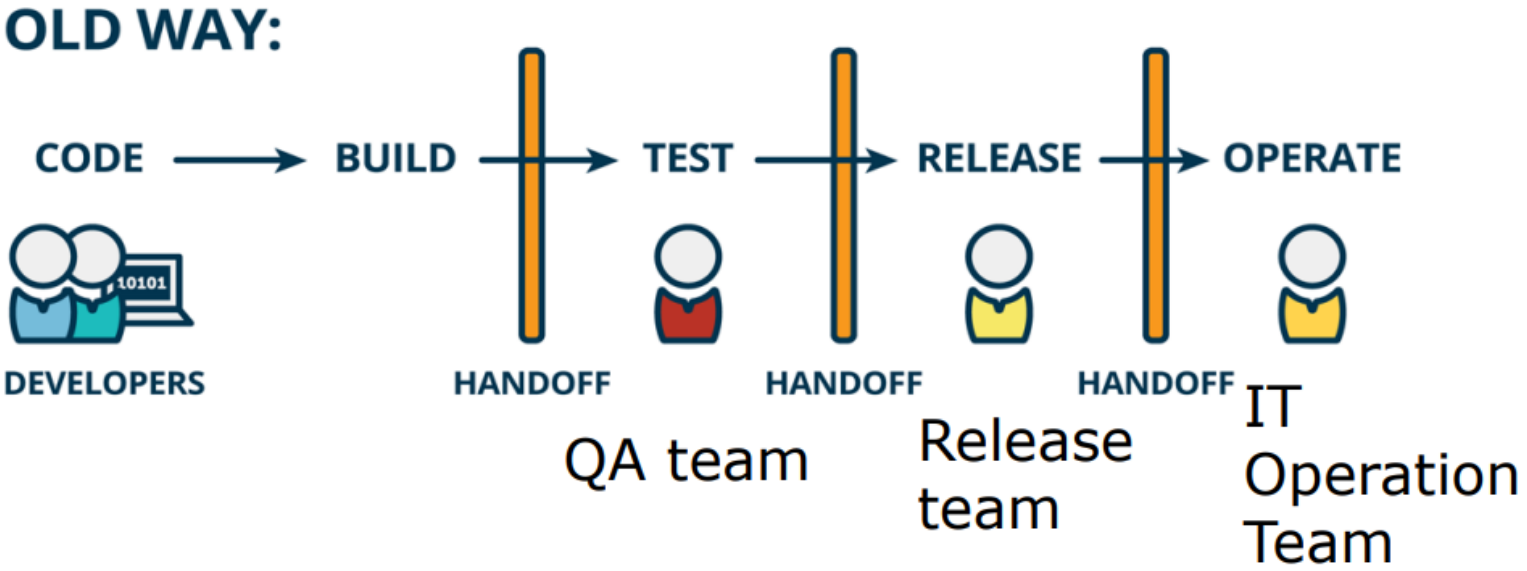
- Software Engineering plays a critical part in the software development
  - Planning
  - Requirements analysis
  - Design
  - Implementation
  - Verification & Validation
  - Maintenance and evolution



Traditional models

- Development team (DEV)
  - Design + Code + Test
- IT Operations Team (OPS)
  - Deploy + Support, Sys Admin + Maintenance
- Wall of confusion (more details)
  - separate entities within an organization
  - different mindsets/objectives
    - OPS : work in a cost-efficient way
    - DEV : efficiently deliver features
  - different tools & environments

Traditional models



Traditional models

- Problem 1: different objectives
  - IT Operations team like the software to be as much stable as possible.
    - might see each change and every new release as a potential danger to stability
  - The Development team is paid for changing and adding new features the software application.
  - Might lead to conflicts between Dev and Ops departments.

Traditional models

- Problem 2: different perspectives
  - Operations team views the application as a black box. They can monitor it with suitable tools (e.g. CPU utilization, I/O load, kernel behaviour)
  - Development team has access to source code and knows the internal working of the application.

Traditional models

- Problem 3: different environments
  - Operations team run the application in the production environment
  - Developers run and test their application in their own computer
  - The two environments may be vastly different in terms of hardware, software, libraries, etc.

Traditional models

- Problem 4: driven by different requirements
  - Operations is driven by non-functional requirements (availability, reliability, system speed ...)
  - Development is driven by functional requirements (features).

- Problem 5: client perspective
  - who is really responsible?

Traditional models

- Problem 6: combining know-how
  - Operations and development have only their own part of knowledge and the required tools to build, test and run the application
  - A smooth operation of applications is only possible with the combined know-how of operations and development.

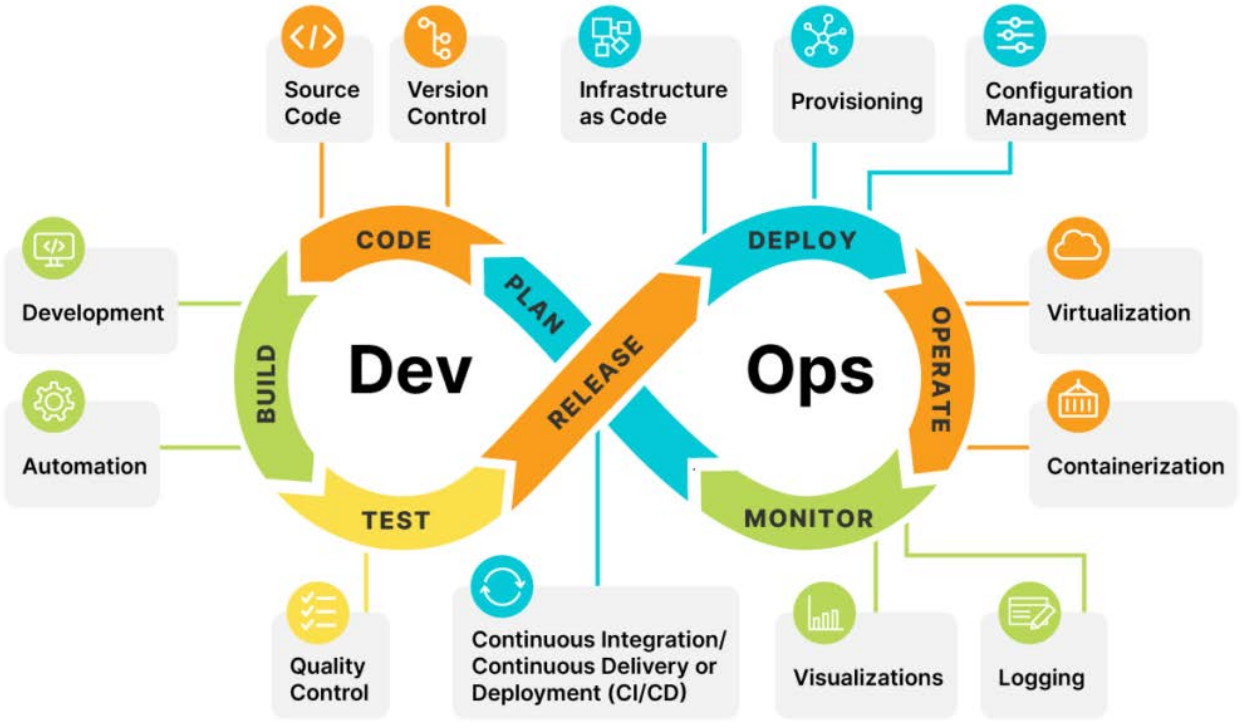
What is DevOps?

- DevOps is a software engineering methodology which aims to unify software development (Dev) and IT operation (Ops)
- Development and operations teams are no longer separated (i.e. no confusion wall)
  - these two teams can be merged into a single team
  - the DevOps engineers work across the entire application lifecycle, from development and test to deployment to operations.

DevOps

- Collaboration, trust, and shared responsibility
- Support autonomous teams
  - better manage unplanned work
  - release faster and work smarter
  - value feedback

DevOps lifecycle



https://orangematter.solarwinds.com/2022/03/21/what-is-devops/

DevOps lifecycle

- Development (plan, code & build)
  - Agile, incremental and iterative development
- Test
  - Test-driven development, acceptance testing
- Integration (release)
  - New functionality is integrated with existing code, and testing takes place
- Deployment (deploy & operate)
  - Deployment process takes place continuously
- Monitor
  - Bugs and performance reporting in real time

DevOps Vs CI/CD

- DevOps facilitates the implementation of CI/CD:
  - Development knows:
    - how the application has to be configured
    - metrics for monitoring
  - Operations knows:
    - the internal structure of the application
  - Feedback from operation can be used directly to optimize further development of the application (since all necessary roles are united in one team)

DevOps Vs Agile

Features	DevOps	Agile
Agility	both Development & Operations	only Development
Processes	processes such as CI, CD, CT, etc	practices such as Agile Scrum, etc.
Key Focus	Timeliness & quality have equal priority	Timeliness is the main priority
Source of Feedback	from self (Monitoring tools)	from customers
Scope of Work	Agility & need for Automation	Agility only

DevOps benefits

- Rapid delivery: innovate and improve your product faster (e.g. release new features and bug fixes quickly) => respond to your customers' needs and build competitive advantage.
- Reliability: ensure the quality of application updates and infrastructure changes.
- Scale: Operate and manage your infrastructure and development processes at scale.

DevOps: A Netflix Casey Study

- Edge Engineering is responsible for the first layer of AWS services (Amazon Web Services) that must be up for Netflix streaming to work.
- Releasing a new feature meant devs coordinating with the ops team on things like metrics, alerts, and capacity considerations, and then handing off code for the ops team to deploy and operate.
- To be effective at running the code and supporting partners, the ops teams needed ongoing training on new features and bug fixes.
- Less developer interrupts when things were going well.

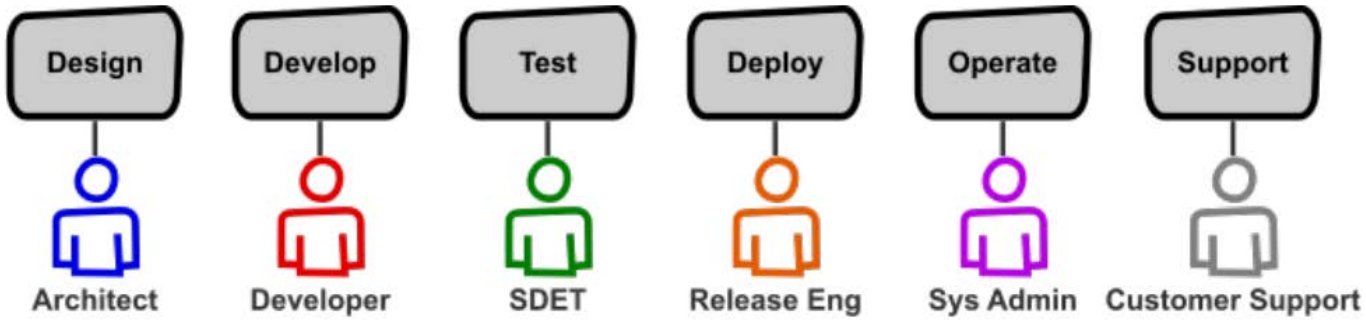
https://netflixtechblog.com/full-cycle-developers-at-netflix-a08c31f83249

DevOps: A Netflix Casey Study

- When things didn’t go well, the costs added up
  - communication and knowledge transfers between devs and ops were messy
  - requiring additional round trips to debug problems or answer partner questions
  - deployment problems had a higher time-to-detect and time-to-resolve due to the ops teams having less direct knowledge of the changes being deployed
  - the gap between code complete and deployed was much longer, with releases happening on the order of weeks rather than days.
  - feedback went from ops, who directly experienced pains such as lack of alerting/monitoring or performance issues and increased latencies, to devs, who were hearing about those problems second-hand.

DevOps: A Netflix Casey Study

- Developing and running a software service involves a full set of responsibilities.
- We had been segmenting these responsibilities. At an extreme, this means each functional area is owned by a different person/role:



DevOps: A Netflix Casey Study

- These specialized roles create efficiencies within each segment while potentially creating inefficiencies across the entire life cycle
  - Specialists develop expertise in a focused area and optimize what's needed for that area
    - Yet, Software requires the entire life cycle to deliver value to customers.
    - Focusing on a slice of the life cycle can create silos that slow down end-to-end progress
  - Grouping differing specialists together into one team may help
    - Yet, communication overhead, introduces bottlenecks, and inhibits the effectiveness of feedback loops

DevOps: A Netflix Casey Study

- Edge Engineering optimize for learning and feedback by breaking down silos and encouraging shared ownership of the full software life cycle (no individual for an area)

- “Operate what you build”
  - having the team that develops a system also be responsible for operating and supporting that system.

DevOps: A Netflix Casey Study

- “Operate what you build”
  - each development team owns deployment issues, performance bugs, capacity planning, alerting gaps, partner support, and so on.
  - direct feedback loops and aligns incentives
  - teams are empowered to make changes to the system design or its underlying code
  - take ownership of the work (be responsible and accountable), rather than relying on external parties or waiting for permission

DevOps: A Netflix Casey Study

- Centralized teams
  - with the mission of developing common tooling and infrastructure (reusable building blocks) to solve problems that every development team has
  - other teams can leverage the pre-built components provided by the centralized teams
    - ensure consistency, standard and quality

DevOps: A Netflix Casey Study

- Empowered with these tools in hand, development teams can
  - decide if their need is important enough for them to solve on their own
  - focus on solving specific problems within their specific product domain
  - centralized teams assess whether the needs are common across multiple dev teams
  - decide if local needs are too specific to warrant centralized investment

- One team, equipped with amazing developer productivity tools, is responsible for the full software life cycle
  - design, development, test, deploy, operate, and support

DevOps: A Netflix Casey Study

- Moving to a full cycle developer model requires a mindset shift
  - Before
    - design+development, and sometimes testing, as the primary way that they create value
    - operations as a distraction, favoring short term fixes to operational and support issues so that they can get back to their “real job”
  - Now
    - use the software development expertise to solve problems across the full life cycle.
    - create software, write test cases, and still other times automate operational aspects

DevOps: A Netflix Casey Study

- With the full cycle model, priority is given to a larger area of ownership and effectiveness in those broader domains
- Breadth requires both interest and aptitude in a diverse range of technologies
  - enjoy and welcome the broader responsibilities
- Developers prefer focusing on becoming world class experts in a narrow field may be uncomfortable
  - support them in finding those roles

DevOps: A Netflix Casey Study

- Breadth increases workload and means a team will balance more priorities every week than if they just focused on one area
- Solution: having an on-call rotation where developers take turns handling the deployment + operations + support responsibilities
  - when done well, that creates space for the others to do the focused, flow-state type work
  - when not done well, teams devolve into everyone jumping in on high-interrupt work like production issues, which can lead to burnout.

DevOps: A Netflix Casey Study

- Conclusion
  - Edge Engineering, whose earlier experiences motivated finding a better model, is actively applying the full cycle developer (design+deploy+support) model
  - Deployments are routine and frequent, canaries take hours instead of days, and developers can quickly research issues and make changes
    - no need to bounce the responsibilities across teams
  - Other groups are seeing similar benefits

DevOps in practice

- Some extra demos of how DevOps is used in major companies:
  - Atlassian: https://youtu.be/neySeor_vAk
  - Netflix: https://youtu.be/UTKIT6STSVM
  - Google: https://youtu.be/OSU8-HJtYYc

References and acknowledgement: E. Wolff, A Practical Guide to Continuous Delivery, Addison-Wesley, 1st edition (February 17, 2017)

Tools used in DevOps

- Source Code Repository: developers check-in and change code => a major component of continuous integration
  - E.g. Git, Subversion, Cloudforce, Bitbucket and TFS
- Build Server: automatically compiles the code in the source code repository into executable code base
  - E.g. Jenkins, SonarQube and Artifactory
- Configuration Management: defines the configuration of a server or an environment.
  - E.g. Puppet and Chef.
- Virtual Infrastructure: programmatically create new virtual machines – provided by cloud vendors
  - E.g. Amazon Web Services and Microsoft Azure
- Test Automation: automatically perform all tests
  - E.g. Selenium and Water